

AI Course

Capstone Project Action Plan

For students (instructor's review required)

© 2025 SAMSUNG. All rights reserved.

Samsung Electronics Corporate Citizenship Office holds the copyright of this document.

This document is a literary property protected by copyright law so reprint and reproduction without permission are prohibited.

To use this document other than the curriculum of Samsung Innovation Campus, you must receive written consent from copyright holder.

Course	AI Course
Team Name	Cogni
Team Leader/ Members	Marwan Elamami (Team Leader) / Abdulrahman Khengari, Ahmed Alaeb, Sanad Ali, Mohammed Thabet, Abdul Majid
Project Title	Bayan: An AI Reading Assistant for Arabic Readers with Dyslexia
Goal	• Help Arabic readers with dyslexia read and understand Arabic text on their own. • Simplify complex Arabic text automatically without changing its meaning. • Add full diacritics (tashkeel) so every word can be pronounced correctly. • Read the text aloud with synchronized word-by-word highlighting. • Deliver this inside the apps people already read in, as an Android text-selection plugin. • Run every model on the device, so the reader's text never leaves the phone and the tool works offline.
Abstract	Arabic creates specific barriers for readers with dyslexia: short vowels are usually omitted, prefixes and suffixes attach directly to words, and long sentences chain many clauses together. Most dyslexia tools target Latin-script languages; we found no deployed Arabic dyslexia-support software. Bayan is a three-stage on-device pipeline that simplifies complex Arabic text with a fine-tuned Arabic encoder-decoder Transformer, restores its diacritics, and reads it aloud with synchronized word highlighting. It is delivered as an Android plugin registered on the system PROCESS_TEXT intent, so “تبسيط” appears in the text-selection menu of every app. The reader is helped where the difficulty actually occurs — no copying, no pasting, no separate app to open. Every claim in this plan is labelled measured, decided or planned. Measured to date: a copy-the-input baseline on two locked test sets, a first simplification model scored against it, a four-model diacritization benchmark, and 9.3% test-set contamination detected and removed from a published Arabic simplification corpus before any training use.
Method	1. Data construction (planned, in progress): easy BAREC sentences and religious scripture are kept unchanged. For each hard sentence an LLM generates several candidate simplifications; an LLM judge calibrated on human annotations scores meaning preservation, and a fine-tuned Arabic BERT classifier checks readability. The best passing candidate is kept, and annotators review samples. 2. Simplification (decided): fine-tune Arabic encoder-decoder Transformers — AraT5v2-base-1024 (368M parameters) and AraBART (139M) — on identical data, and compare them before choosing one to compress. Training uses level tokens so a single model produces three simplification strengths plus a “leave unchanged” mode, because knowing when not to simplify is part of the task.

3. Diacritization (decided, measured): Libtashkeel, selected from a four-model benchmark on the Tashkeela test split — DER 6.91% excluding case endings, 4.79 MB, 125 sentences/sec, MIT licence, exports to ONNX. CAMEL Tools was rejected on a harder ground than accuracy: it altered base letters in 32% of sentences, corrupting the text before the simplifier sees it.
4. Read-aloud (decided): a pretrained Arabic text-to-speech model with word timestamps for synchronized highlighting.
5. Evaluation (measured): every score is reported beside a copy-the-input baseline on the same split, because on a corpus containing many already-simple sentences a model that does nothing can score well. Test sets are locked and recorded by SHA-256 in a committed manifest, and a leakage check runs over every training file before it is used. Metrics: SARI, BLEU, readability reduction measured by an independent BAREC model, BERTScore for meaning preservation, DER/WER for diacritization, and human error annotation.
6. Application (decided): one Android plugin on the PROCESS_TEXT intent with a Material 3 bottom sheet, ONNX Runtime Mobile for every model, greedy decoding, and the model downloaded on first run to keep the install small.

Data	
Sources • BAREC (CAMEL Lab, CC BY-SA 4.0): 69,441 Arabic sentences with readability labels — source sentences to simplify, and the labels for the readability classifier. • SAMER Simplification Corpus v1 (CAMEL Lab): 14,343 training pairs of complex-to-simple Arabic at three levels. CAMEL Lab approved fine-tuning on it and publishing the resulting weights for non-commercial use; the corpus itself is never redistributed. • Baseet (University of Jeddah): 34,294 sentences with three LLM-generated simplification levels. 3,190 rows (9.3%) were found to overlap our locked test sets and were removed before any training use; 31,104 remain. • DAASI (CC0): 1,509 human-written simplification pairs of government and insurance text, plus a 350-pair held-out test set — our only out-of-domain test set with human-written references. • Bayan simplification corpus (built by the team): complex-to-simple pairs generated with LLM APIs, filtered by the validators and reviewed by team annotators. • Tashkeela: fully diacritized Arabic text for validating the diacritization stage. How we use it Training the simplification models, calibrating the validators and the readability classifier, validating diacritization, and benchmarking Bayan against published baselines. Two splits are locked, hashed and never trained on: BAREC test (7,286 rows) and SAMER test (3,277 rows).	
Expected Outcome	
Deliverables • A working Android plugin: the reader selects Arabic text in any app and receives a simplified, fully diacritized version, read aloud with synchronized word highlighting. • Fine-tuned simplification model(s) quantized to run on-device, benchmarked against a copy-the-input baseline, a published system and large language models. • A new Arabic simplification corpus, an evaluation benchmark, and a reusable test-set leakage check. Results to date (measured) • Copy-the-input baseline established on both locked test sets: SARI 77.5 overall on SAMER test, but 56.1 on the rows a human actually simplified — the bar any model must clear. • First model fine-tuned on SAMER and evaluated. It does not yet clear that baseline under the greedy decoding we ship on-device; the measured gap set the training recipe for the next runs. • Diacritization benchmark complete across four models: DER, WER, size, speed and licence. • 9.3% test-set contamination detected and removed from a published Arabic corpus before use. Benefits • Greater reading independence and comprehension for Arabic readers with dyslexia. • Support for students, teachers and schools, and for readers with low literacy and learners of Arabic. • Everything runs on the device, so it works offline and the reader's text stays private.	
Role by Member	

Marwan Elamami | Team Leader & On-Device Model Lead

Project management; system architecture; model compression and quantization for on-device use; integration.

Abdulrahman Khengari | Data Lead

Building and validating the simplification corpus; readability classifier; data quality control; annotation guidelines.

Ahmed Alaeb | Model Training Lead

Training and evaluating the simplification models; corpus conversion; training infrastructure.

Sanad Ali | Machine Learning Engineer

Diacritization model selection and benchmarking; co-training the simplification models.

Mohammed Thabet | Application Engineer

Android plugin development; on-device inference integration; read-aloud with word highlighting.

Abdul Majid | Evaluation & Benchmarking Lead

Test-set design and locking; leakage detection; the scoring pipeline; automatic and human evaluation.

Schedule Summary	• <i>Week 1 (Wed 16 – Sun 20 Sep) — complete: data pipeline and validators; readability classifier; test-set design; product-form decision; application skeleton.</i> • <i>Week 2 (Mon 21 – Sun 27 Sep) — in progress: corpus generation and annotation; first and second simplification training runs; diacritization benchmark; scoring pipeline and locked test sets; Android plugin with on-device inference.</i> • <i>Week 3 (Mon 28 Sep – Sun 4 Oct): architecture selected and frozen; int8 quantization with size and latency measured on a real device; full benchmark including human error annotation; end-to-end integration with read-aloud and highlighting; user testing with dyslexic readers.</i> • <i>Week 4 (Mon 5 – Tue 6 Oct): final report and demo video for submission on Mon 5 Oct; final pitch on Tue 6 Oct.</i> <i>Progress is tracked work-package by work-package in the Work Breakdown Structure: 8 phases, 38 work packages, one named owner each.</i>
Comment & Assessment	◁Comment and assessment by the instructor.▷